

1- Qu'est-ce que Node.js ?

- C'est un **environnement d'exécution JavaScript** qui permet d'exécuter du JavaScript **hors du navigateur**.
- Basé sur le moteur **V8 de Google Chrome**, il rend JavaScript très rapide.
- Sert à créer :
 - des **serveurs web**
 - des **APIs**
 - des **scripts** et outils en ligne de commande
 - des **applications temps réel** (chat, jeux, etc.)

Exemple concret : quand tu vas sur un site e-commerce, le front (HTML, CSS, React) envoie une requête au backend (Node.js) pour récupérer les produits.

2- Pourquoi utiliser Node.js ?

- **Un seul langage (JS)** pour le front et le back → facile pour apprendre.
 - **Asynchrone & non bloquant** → Node gère plusieurs requêtes en même temps (idéal pour API).
 - **Énorme communauté** → beaucoup de bibliothèques disponibles via [npm](#).
-

3- Modules Node.js

Un **module** est comme une boîte à outils que l'on importe.

Types :

1. **Modules natifs** (déjà inclus dans Node.js)
 - [fs](#) → travailler avec des fichiers

- `http` → créer un serveur
- `os` → infos sur le système

2. Modules externes (via `npm`)

- Exemple : `express`, `mongoose`, `lodash`

3. Modules personnalisés

- Créés par toi, exportés/importés entre fichiers

4- Créer un serveur avec Node.js (bas niveau)

- Node peut créer un serveur avec le module `http`.
- Mais gérer les routes, les requêtes et les réponses devient vite compliqué.

C'est pour ça qu'on utilise **Express.js**.



5- Qu'est-ce que Express.js ?

- C'est un **framework pour Node.js**.
- Il simplifie la création de serveurs web et d'APIs.
- Avec Express, tu peux :
 - Créer facilement des routes (`GET`, `POST`, `PUT`, `DELETE`)
 - Gérer les données envoyées en **JSON**
 - Ajouter des **middlewares** (authentification, logs, validation, etc.)

6- Concepts clés Express.js

a) Application Express

- C'est ton serveur.
 - Tu crées une app avec :

```
const express = require("express");
```
 - ```
const app = express();
```
- 

### b) Route

- Une route définit **quelle réponse envoyer selon une requête HTTP**.
  - Exemple :

```
app.get("/hello", (req, res) => {
```
  - ```
  res.send("Hello Express !");
```
 - ```
});
```
- 

### c) Middleware

- C'est une fonction intermédiaire qui intercepte la requête avant qu'elle arrive à la route.
  - Exemple : pour analyser du JSON

```
app.use(express.json());
```
- 

### d) CRUD avec Express

- Create → **POST**
- Read → **GET**
- Update → **PUT**
- Delete → **DELETE**

Exemple :

- `app.post("/products", (req, res) => {`
- `res.send("Produit ajouté");`
- `});`

---

## 7- Node.js + Express.js = API REST

Avec Node et Express, tu peux créer une **API REST**, c'est-à-dire un ensemble de routes qui permettent :

- D'ajouter des données (produits, utilisateurs, etc.)
- De lire ces données
- De les modifier
- De les supprimer
  -